

Klasy w języku Java

Zbigniew Marszałek

Klasa w języku Java

Podstawowymi pojęciami w językach zorientowanych obiektowo jest klasa i obiekt. Klasy możemy rozumieć jako schematy, za pomocą których konstruujemy obiekty. W języku Java wszystko z czym się spotykamy bazuje na klasach, co stanowi podstawę programowania obiektowego. Każda klasa składa się z danych i metod składowych takich jak konstruktory, operatory właściwości, metody i zdarzenia. W przypadku tworzenia obiektu klasy korzystamy z konstruktora wywoływanego za pomocą polecenia new. Wszystkie klasy w języku Java dziedziczą po wspólnej klasie bazowej Object. Dziedziczenie to ma charakter domyślny i niejawny dlatego nie wymaga dokonywania dodatkowych deklaracji przy tworzeniu klasy.

Programowanie zorientowane obiektowo (ang. *Object Oriented Programming OOP*) bazuje na budowie obiektów posiadających pewne cechy i konstrukcji metod dokonujących zmian w istniejących obiektach lub tworzących nowe obiekty. Konwencjonalne oprogramowanie strukturalne polegało na zaprojektowaniu funkcji działających w oparciu o algorytmy, które rozwiązują postawione zadanie. W programowaniu obiektowym podstawowym zagadnieniem jest zdefiniowanie sposobu przechowywania danych, a dopiero później poszukiwania algorytmów umożliwiających przekształcania danych w żądany sposób.

Kluczem do uzyskania dużej wydajności w OOP jest powiązanie każdego obiektu z metodami wykonującymi operacje na danym obiekcie. W szczególności, obiekt nie powinien bezpośrednio manipulować danymi należącymi do innego obiektu, ani też nie powinien zezwalać na manipulowanie innym obiektom na swoich danych. Całość komunikacji powinna odbywać się za pomocą zdefiniowanych metod. Ukrywanie danych jest podstawowym zagadnieniem programowania obiektowego i nosi nazwę enkapsulacji. Enkapsulacja jest powiązaniem danych pomiędzy sobą i sposobem korzystania z nich w ramach pojedynczego pakietu. Dzięki enkapsulacji danych zwiększamy elastyczność programu i zmniejszamy ryzyko powstania błędów przy projektowaniu aplikacji.

Definicja klasy

Definicja klasy składa się z opcjonalnej listy atrybutów, opcjonalnego modyfikatora, słów kluczowego `class` albo słów kluczowych `static class` oraz opcjonalnej listy klas bazowych lub interfejsów wykorzystywanych do tworzenia danej klasy. Po liście klas bazowych zapisany zostaje kod klasy (ciało klasy), które zawiera między innymi właściwości metody. Przedstawmy prostą klasę przechowującą informacje identyfikatora użytkownika, jego nazwisko i telefon. Kod definicji klasy i programu tworzącego na jej podstawie obiekty może wyglądać następująco:

```
package b2a;
import java.util.*;
class Dane{
    private int id;
    private String nazwisko, telefon;
    void seti(int id){
        this.id = id;
    }
    int geti(){
        return id;
    }
    void setn(String nazwisko){
        this.nazwisko = nazwisko;
    }
}
```

```
String getn() {  
    return nazwisko;  
}  
void sett(String telefon) {  
    this.telefon = telefon;  
}  
String gett() {  
    return telefon;  
}  
public Dane() {}  
public Dane(int id, String nazwisko, String telefon) {  
    this.id = id;  
    this.nazwisko = nazwisko;  
    this.telefon = telefon;  
}  
}
```


Działanie programu

- Tworzymy jest obiekt klasy Dane.
- Nadajemy wartości polom obiektu za pomocą przeciążonego konstruktora klasy.
- Drukujemy wartości pól utworzonego obiektu.
- Zmieniamy wartości pól obiektu za pomocą odpowiednich właściwości klasy dane.
- Drukujemy właściwości pól po zmianie.



```
run:  
1 - Kowalski +48505673287  
3 - Nowak +49234346329
```

Modyfikatory dostępu

Podstawową rolą modyfikatorów jest określenie zasięgu (widoczności) typów i składowych typów. W zastosowaniu do klas modyfikatory określają z jakiego poziomu będą one widoczne (z poziomu zestawu, z poziomu klasy zawierającej lub z poziomu klasy potomnej).

Rozróżniamy podstawowe typy modyfikatorów:

- `public` – klasa jest dostępna dla dowolnego innego kodu.
- `protected` – dostęp ograniczony do klasy zawierającej lub klasy potomnej.
- `private` – dostęp ograniczony tylko do samej klasy.

Modyfikator `protected` ma zastosowanie w przypadku klas zagnieżdżonych tzn. zdefiniowanych wewnątrz danej klasy.

Jeśli nie zastosuje się żadnego modyfikatora, składowa klasy jest dostępna publicznie dla wszystkich innych klas swojego pakietu.

Składowe klasy

Zmienne i metody definiowane w klasie nazywane elementami klasy. Modyfikatory dostępu stosowane w deklaracji klasy mogą być stosowane do elementów klasy. Za pomocą modyfikatorów określamy, do których składowych będą miały dostęp inne klasy. Do działania obiektu niezbędne są dane, które mogą być wykorzystywane przez metody związane z obiektem np. dane osoby zapisane w obiekcie. Metody są blokami instrukcji zdefiniowanymi w ramach klasy i umożliwiają wykonanie określonych operacji na danych klasy.

Pola i metody statyczne

W języku Java słowo kluczowe *static* w deklaracji pola oznacza zmienną związaną z klasą – taką samą dla wszystkich tworzonych obiektów tej klasy. W przypadku deklarowania stałych dodajemy słowo kluczowe *final*, które oznacza, że stała ta nie może zostać zmieniona podczas działania programu.

Metody statyczne to metody, które nie operują na obiektach. Przykładowo metoda *sqrt* klasy *Math* jest metodą statyczną obliczającą pierwiastek z liczby rzeczywistej i nie wykonuje operacji na polach klasy.

Poniżej przedstawiono przykład metody statycznej.

```
package b2a1;
import java.math.*;
class Miara{
    private static final double p = Math.PI/180.0;
    public static final double radian(double stopien){
        double rad = p*stopien;
        return rad;
    }
}
public class B2a1 {
    public static void main(String[] args) {
        double st = Miara.radian(180.0);
        System.out.println("PI = " +st);
    }
}
```



```
run:
```



```
PI = 3.141592653589793
```

W programie dostęp do zmiennej `p` został uzyskany za pomocą metody `radian`, która zamienia stopnie na radiany. Istotną zasadą jest to, że wartość stałej jest ustalona w momencie kompilacji i nie może ulec zmianie podczas działania programu.

Pola

Pola wykorzystujemy do przechowywania danych wewnątrz klasy. W celu zabezpieczenia obiektu przed ingerencją z zewnątrz, pola powinny nosić atrybut **private**. Jeżeli dostęp do danych prywatnych z zewnątrz jest konieczny używamy odpowiednich modyfikatorów umożliwiających dokonywanie pożądaných operacji. Poza przedstawionymi poprzednio modyfikatorami dostępu do pól obsługiwany jest jeszcze modyfikator **static**. Oznacza on, że pole jest częścią stanu klasy, a nie tworzonych obiektów i nie można odwołać się do pola za pomocą konstrukcji

nazwa_klasy.nazwa_pola.

Metody operujące na polach

Metody są wykorzystywane do kontrolowania dostępu do danych zapisanych wewnątrz klasy. W przedstawionym przez nas przykładzie klasy danych osobowych, zastosowaliśmy metody ustawiającą i odczytującą pola identyfikujące rekord.

```
void setn(String nazwisko) {  
    this.nazwisko = nazwisko;  
}  
String getn() {  
    return nazwisko;  
}
```

Składnia dostępu do pola

```
a.seti(3);  
a.setn("Nowak");  
a.sett("+49234346329");  
System.out.printf(lp,"%d - %s %s%n",  
                    a.geti(),a.getn(),a.gett());
```

Blok metody **get** umożliwiają odczyt pól, natomiast metody **set** odpowiadają za ustawianie wartości pól obiektu. Wszystkie instrukcje return zwracają wyrażenie, które można przekonwertować na żądany typ.

Liczby zespolone

Zaprojektujmy klasę obsługującą liczby zespolone. Liczba zespolona jest parą liczb rzeczywistych typu **double**. Jeżeli liczby te zostaną zadeklarowane jako prywatne, musimy skonstruować właściwości umożliwiające zapis i odczyt składowych liczby zespolonej. Nasz program może wyglądać następująco:

```
class Liczba{  
    private double x, y;  
    // Pola reprezentujące liczbę zespoloną.  
    double getX() {  
        return x;  
    }  
    double getY() {  
        return y;  
    }  
    // Metoda umożliwiające odczyt liczby zespolonej.
```

```
void setx(double x) {
    this.x = x;
}
void sety(double y) {
    this.y = y;
}
// Metoda umożliwiające nadawanie wartości
// liczbie zespolonej.
}
public class B2a2 {
    public static void main(String[] args) {
        Liczba p=new Liczba();
        // Konstrukcja liczby zespolonej za pomocą konstruktora
        // domyślnego.
        p.setx(2.1);
        p.sety(-3.4);
        // Nadanie wartości liczbie zespolonej.
```

```
        System.out.printf("(x; y) = (%g; %g)%n",  
                           p.getx(), p.gety());  
// Wydruk liczby zespolonej.  
    }  
}
```

Przedstawiony program zapewnia dostęp do pól liczby zespolonej za pomocą odpowiednich właściwości i wydruk liczby jest następujący:



run:



(x; y) = (2,10000; -3,40000)

Metody i przekazywane parametry

Funkcja zdefiniowana wewnątrz klasy nosi nazwę metody. Metodę wewnątrz klasy definiuje się za pomocą składni

[modyfikator dostępu]

zwracana_wartość nawa_metody(argumenty)

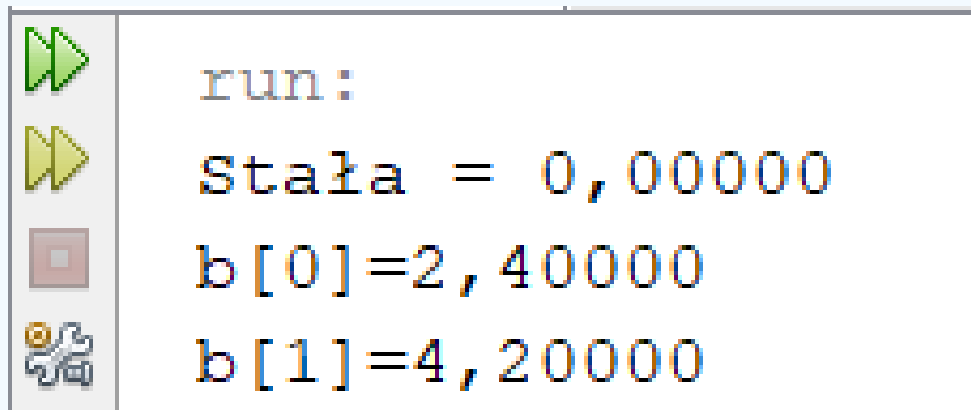
Jeżeli metoda nie zwraca żadnej wartości, zwracaną wartość definiujemy jako void. Wartości do metody przekazujemy przez argumenty. Typy proste są przekazywane przez kopiowanie wartości z metody wywołującej do metody wywołanej. Oznacza to, że ich modyfikacja wewnątrz metody nie powoduje zmiany zmiennych w metodzie wywołującej. W języku Java nie ma możliwości działania bezpośrednio na zmiennych wywoływanych przez metodę!

Obiekty są przekazywane za pomocą referencji i dokonywane zmiany w wywoływanej metodzie są widoczne w obiekcie w wywołującej metodzie. Poniższy przykład pokazuje, że zmiana wartości zmiennej prostej nie ulega zmianie po wywołaniu metody bez względu na dokonywane zmiany.

Natomiast zmiana dokonana w obiektach jest widoczna w funkcji Main. Tak samo zmiana wartości elementów tablicy jest widoczna w funkcji Main.

```
package b2b;  
class Cena{  
    void stala(double a) {  
        a = 2.2;  
    }  
    // Zmiana wartości zmiennej.  
    void tablica(double a[]) {  
        for (int i = 0; i < a.length; i++)  
            a[i] *= 2.0;  
        //Powiększenie wszystkich elementów  
        //tablicy dwukrotnie.  
    }  
}
```

```
public class B2b {  
    public static void main(String[] args) {  
        Cena c = new Cena();  
        // Konstrukcja klasy.  
        double a = 0.0;  
        //Deklaracja zmiennej i nadanie wartości początkowej zero.  
        c.stala(a);  
        // Wywołanie metody stala z klasy Cena.  
        System.out.printf("Stała = %g%n", a);  
        // Wartość zmiennej nie ulega zmianie.  
        double b[] = { 1.2, 2.1 };  
        // Deklaracja tablicy i nadanie wartości początkowych.  
        c.tablca(b);  
        //Wywołanie metody dokonującej zmiany wartości elementów.  
        for (int i = 0; i < b.length; i++)  
            System.out.printf("b[%d]=%g%n", i, b[i]);  
        // Wydruk tablicy po zmianie wartości.  
    }  
}
```



```
run:
Stała = 0,00000
b[0]=2,40000
b[1]=4,20000
```

Bezpośrednio metoda działa na zmiennej przekazywanym przez metodę wywołującą. Dlatego nie może dokonywać zmiany elementów w przekazanej tablicy i nie może dokonać zmiany wymiaru tej tablicy ani jej zredefiniować. Może natomiast dokonać zmiany wartości tablicy, gdyż tablica jest przekazana przez referencje. Gdy przekażemy jednak obiekt do metody sytuacja zmienia się radykalnie, ponieważ przekazywanie obiektów odbywa się przez referencje. Prezentuje to następny przykład:

Metoda może przekazywać referencję obiektu

```
package b2b1;
class Liczba{
    double x, y;
    Liczba(double x, double y) {
        this.x = x;
        this.y = y;
    }
    void zamiana(Liczba a) {
        a.x *= 2.0;
        a.y *= 3.0;
    }
}
```

```
public class B2b1 {  
    public static void main(String[] args) {  
        Liczba d = new Liczba(1.0,1.0);  
        System.out.printf("( %g; %g)%n", d.x,d.y);  
        d.zamiana(d);  
        System.out.printf("( %g; %g)%n", d.x,d.y);  
    }  
}
```





```
run:  
( 1,000000; 1,000000)  
( 2,000000; 3,000000)
```

Metoda może zwracać referencję do obiektu

```
class Tabela{
public double[] nowa() {
    double [] a = { 1.2, 2.3, 3.4 };
    // Utworzenie tabeli.
    return a;
}}

public class B2b3 {
    public static void main(String[] args) {
        Tabela t = new Tabela();
        // Utworzenie egzemplarza klasy Tabela.
        double [] b = t.nowa();
        // Zwrócenie uchwytu do tabeli.
        for(int i = 0; i < b.length; i++)
            System.out.println(b[i]);
        // Wydruk wartości zapisanych w tablicy.
    }
}
```

	run:
	1.2
	2.3
	3.4

Rekurencja

Java obsługuje rekurencję. Rekurencja polega na odwołaniu się do siebie samego w trakcie wykonywania obliczeń. Klasycznym zadaniem jest obliczenie silni z liczby.

```
class Algorytm{
    int silnia(int n){
        if( n > 1) return n*silnia(n-1);
        else return 1;
    }
}

public class B2b2 {
    public static void main(String[] args) {
        Algorytm at = new Algorytm();
        System.out.println("3! = " + at.silnia(3));
    }
}
```

Klasa liczb zespolonych

```
import java.util.*;
class Liczba{
    public double x, y;
    public Liczba(){}
    public Liczba(double x, double y){
        this.x = x;
        this.y = y;
    }
    public String toString(){
        return String.format("( %g; %g)", x, y);
    }
}
```

```
public String toString() {  
    return String.format("( %g; %g) ", x, y);  
}  
public boolean equals(Object a) {  
// Metoda sprawdzajaca równość liczb zespolonych.  
    if (a == null) return false;  
    if (this.getClass() != a.getClass())  
        return false;  
    Liczba ta = (Liczba) a;  
    return (this.x == ta.x) && (this.y == ta.y);  
}  
public Liczba get() {  
// Metoda kopiujaca obiekt.  
    return new Liczba(x, y);  
}
```

```
        public int hashCode() {  
// Metody zwracająca hashCode.  
            return Objects.hash(x, y);  
        }  
}  
  
public class B2c {  
    public static void main(String[] args) {  
        Liczba p1 = new Liczba(1.2, 2.3);  
// Deklaracja liczby zespolonej.  
        Liczba p2 = p1.get();  
// Wykonanie kopi liczby zespolonej.  
        Liczba p3 = p1;  
// Deklaracja liczby zespolonej na podstawie  
//pierwszej liczby
```

```
Liczba p4 = new Liczba(1.2, 2.3);  
System.out.println(p1.equals(p3)); // true  
System.out.println(p1 == p3); // true  
System.out.println(p1 == p4); // false  
System.out.println("Wartość liczby wynosi:" + p1);  
}
```



run:



true



true



false

Wartość liczby wynosi: (1,20000; 2,30000)

Porównanie wartości i referencji liczb zespolonych

Jako pierwsze są porównywane wartości skopiowanych obiektów liczb zespolonych. Stąd wynik porównania true. Następnie porównywane są referencje liczby zespolonej i skopiowanej liczby zespolonej. Ponieważ zostały one umieszczone w tym samym miejscu na zarządzanej stercie, to wynik porównania referencji jest true.

W przypadku trzeciego porównania referencji obiektów są różne i wynik porównania false. Druga z porównywanych liczb zespolonych jest utworzona za pomocą kreatora i zostaje umieszczona w innym miejscu pamięci.

.

Brak możliwości przeciążenia operatorów

Język Java nie oferuje techniki przeciążenia operatorów (ang. *Operator Overloading*), która umożliwia implementowanie kodu dla danego operatora określając działania na obiektach klasy. Przedstawimy mechanizm konstrukcji metod liczb zespolonych umożliwia wykonywanie operacji na liczbach zespolonych wg. definicji operacji liczb zespolonych. Operacje na liczbach możemy zapisać w następujący sposób:

- Dodawanie: $(a, b) + (c, d) = (a + c, b + d)$
- Odejmowanie: $(a, b) - (c, d) = (a - c, b - d)$
- Mnożenie: $(a, b) * (c, d) = (a*c - b*d, a*d + b*c)$
- Dzielenie: $(a, b) / (c, d) = ((a*c + b*d)/(c*c + d*d), (b*c - a*d)/(c*c + d*d))$

```
class Liczba{
    public double x, y;
    public Liczba(){
        this.x = 0.0;
        this.y = 0.0;
    }
    public Liczba(double x, double y){
        this.x = x;
        this.y = y;
    }
    public String toString(){
        return String.format("( %g; %g) ",x,y);
    }
}
```

```
public Liczba dodaj(Liczba z) {  
    double a = this.x + z.x;  
    double b = this.y + z.y;  
    return new Liczba(a, b);  
}  
  
public Liczba odejmij(Liczba z) {  
    double a = this.x - z.x;  
    double b = this.y - z.y;  
    return new Liczba(a, b);  
}  
  
public Liczba pomnoz(Liczba z) {  
    double a = this.x * z.x - this.y * z.y;  
    double b = this.x * z.y + this.y * z.x;  
    return new Liczba(a, b);  
}
```

```
public Liczba podziel(Liczba z) {  
    double d = z.x*z.x +z.y*z.y;  
    double a = (this.x*z.x+this.y*z.y)/d;  
    double b = (this.y*z.x-this.x*z.y)/d;  
    return new Liczba(a, b);  
}  
  
public boolean equals(Object a) {  
    // Metoda sprawdzajaca równość liczb zespolonych.  
    if (a == null) return false;  
    if (this.getClass() != a.getClass())  
        return false;  
    Liczba ta = (Liczba) a;  
    return (this.x == ta.x) && (this.y == ta.y);  
}  
}
```

```
public class C2d {  
    public static void main(String[] args) {  
        Liczba p1 = new Liczba(1 , 1);  
        // Deklaracja pierwszej liczby zespolonej.  
        Liczba p2 = new Liczba(2 , 2);  
        // Deklaracja drugiej liczby zespolonej.  
        System.out.printf("%s + %s = %s%n",p1,p2,p1.dodaj(p2));  
        // Wydruk dodawania liczb zespolonych.  
        System.out.printf("%s + %s = %s%n",p1,p2,p1.odejmij(p2));  
        // Wydruk odejmowania liczb zespolonych.  
        System.out.printf("%s + %s = %s%n",p1,p2,p1.pomnoz(p2));  
        // Wydruk mnożenia liczb zespolonych.  
        System.out.printf("%s + %s = %s%n",p1,p2,p1.podziel(p2));  
        // Wydruk dzielenia liczb zespolonych.  
    }  
}
```

Operacje na liczbach zespolonych

Do wydruku liczby zespolonej została wykorzystana metoda *printf()*, która dokonuje zamiany liczby zespolonej do łańcucha znaków za pomocą przeciążonej metody *toString()* w klasie *Liczba*. Wydruk wykonanych operacji na liczbach zespolonych wygląda następująco:

run:

```
( 1,00000; 1,00000) + ( 2,00000; 2,00000) = ( 3,00000; 3,00000)
( 1,00000; 1,00000) + ( 2,00000; 2,00000) = ( -1,00000; -1,00000)
( 1,00000; 1,00000) + ( 2,00000; 2,00000) = ( 0,00000; 4,00000)
( 1,00000; 1,00000) + ( 2,00000; 2,00000) = ( 0,500000; 0,00000)
```

Konstruktor

Konstruktor jest metodą, która odpowiada za inicjalizację klasy lub obiektu. Składnia konstruktora jest taka sama jak zwykłych metod. Istnieją jednak podstawowe różnice:

- Nazwa konstruktora musi być identyczna jak nazwa klasy.
- Konstruktor nie zwraca rządnej wartości.
- Jeśli nie zdefiniujemy konstruktora dla budowanej klasy, zostanie automatycznie utworzony bezparametrowy konstruktor domyślny.

Konstruktor

Konstruktor jest wywoływany w momencie tworzenia instancji danej klasy. Ponieważ klasa może zawierać wiele konstruktorów, wywoływany jest automatycznie odpowiedni konstruktor według podanej w kodzie definicji.

W klasie liczb zespolonych zostały zdefiniowane dwa konstruktory, które są używane do inicjalizacji liczb zespolonych.

```
public Liczba () {  
    this.x = 0.0;  
    this.y = 0.0;  
}  
public Liczba(double x, double y) {  
    this.x = x;  
    this.y = y;  
}
```

Klasa zamiast struktury

Pojęcie struktury pochodzi z języka C i określa obiekty złożone z kilku zmiennych różnego typu zapisanych pod jedną nazwą. W wielu językach struktury nazywane są rekordami i uogólnieniem pojęcia struktury jest klasa. W języku Java, który jest językiem w pełni obiektowym, strukturę zastępuje klasa. Niemnie rekordy zapisane za pomocą klasy mogą działać tak samo jak w języku C. Wystarczy zadeklarować wszystkie pola jako publiczne tak samo jak metody potrzebne do wyszukiwania danych. Gdybyśmy pominęli deklaracje dostępu do pól i metod, to w języku Java podobnie jak w języku C są one ogólnodostępne.

Przedstawimy przykład wyszukiwania rekordów w tablicy rekordów zapisanych za pomocą klasy. Nasz rekord zawiera tylko informacje: identyfikator, nazwisko i telefon. Rekordy w tablicy są przeszukiwane liniowo w celu znalezienia telefonu danej osoby.

Program może wyglądać następująco:

Klasa dane numeru telefonu

```
import java.util.*;
class Dane{
    public int id;
    public String nazwisko, telefon;
    public Dane(int id, String nazwisko, String telefon){
        this.id = id;
        this.nazwisko = nazwisko;
        this.telefon = telefon;
    }
}
```

Klasa wyszukiwania rekordu

```
class Rekord{  
    public void szukaj(Dane a[], String s3){  
        for(int i=0; i < a.length; i++)  
            if(s3.compareToIgnoreCase(a[i].nazwisko)==0){  
                System.out.println(a[i].id + " " +  
                                   a[i].nazwisko + " "+a[i].telefon);  
                return;  
            }  
        System.out.println("Nie znalazłem w bazie danych");  
    }  
}
```

```
public class C2f {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in, "Cp1250");  
        Dane a[] = new Dane[4];  
        a[0] = new Dane(1, "Kowalski", "+48505672345");  
        a[1] = new Dane(2, "Baka", "+48500435981");  
        a[2] = new Dane(3, "Nowak", "+48600435980");  
        a[3] = new Dane(4, "Żyrafa", "+496505435980");  
        // Wpisanie rekordów.  
        for (int i = 0; i < a.length; i++)  
            System.out.println(a[i].id + " " +  
                               a[i].nazwisko + " " + a[i].telefon);  
        // Wyświetlenie rekordów w tablicy.
```

```
System.out.print("Podaj nazwisko do znalezienia:");  
String s3 = in.nextLine();  
// Pobranie poszukiwanego nazwiska.  
Rekord re = new Rekord();  
// Umieszczenie klasy na zarządzanej stercie.  
re.szukaj(a,s3);  
// Wywołanie metody wyszukującej osoby w tablicy.  
}  
}
```



run:

1 Kowalski +48505672345

2 Baka +48500435981

3 Nowak +48600435980

4 Żyrafa +496505435980

Podaj nazwisko do znalezienia:żyrafa

4 Żyrafa +496505435980